

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	Sankar.R	Reg. No.:	192525131
Section / Batch:	Slot - B	Date:	02/09/2026

Code of Conduct

I Sankar.R [192525131] (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sankar.R

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

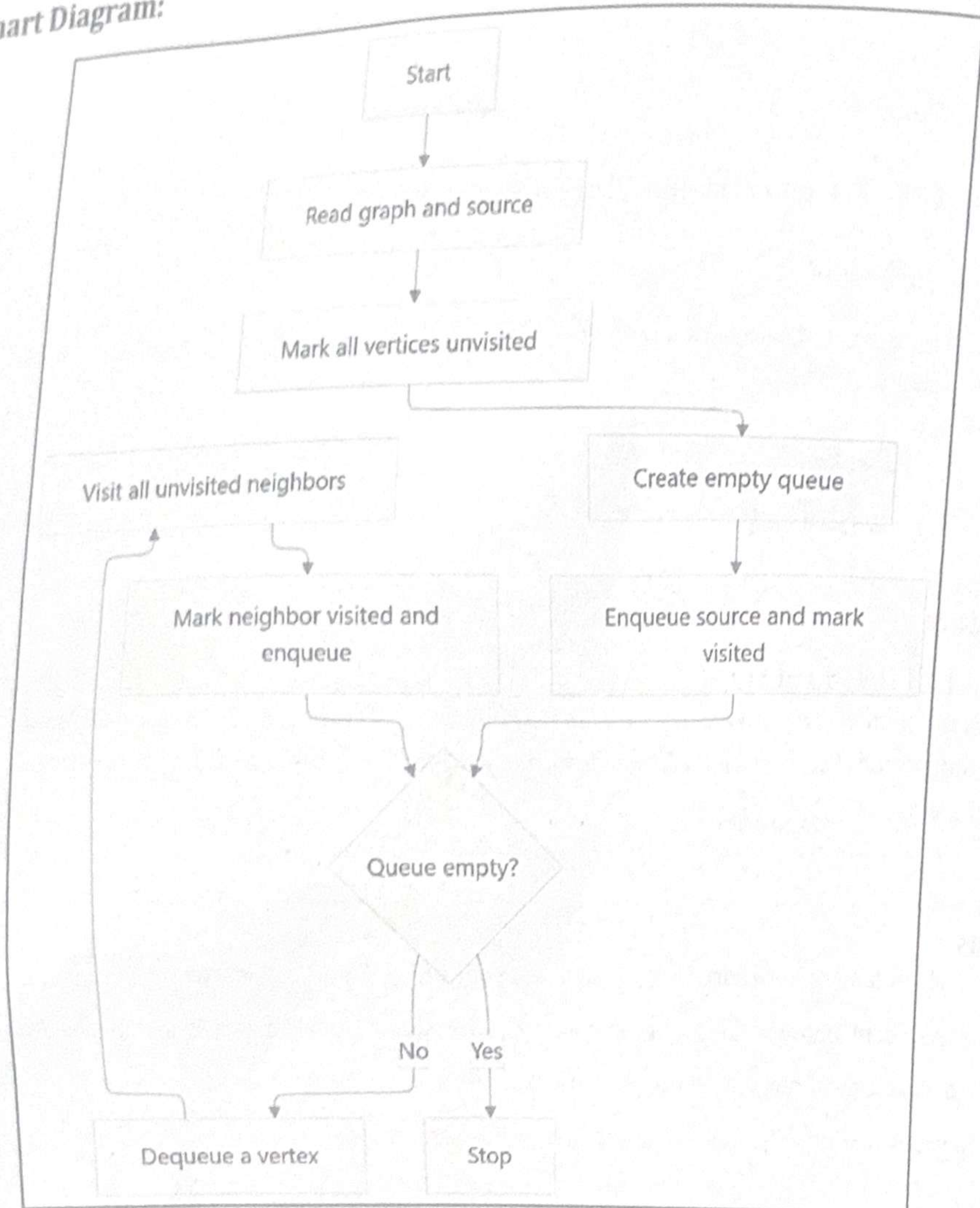
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Flowchart-To-Code Conversion

1. Convert the flowchart for Breadth First Search traversal into code.

(20 Marks)

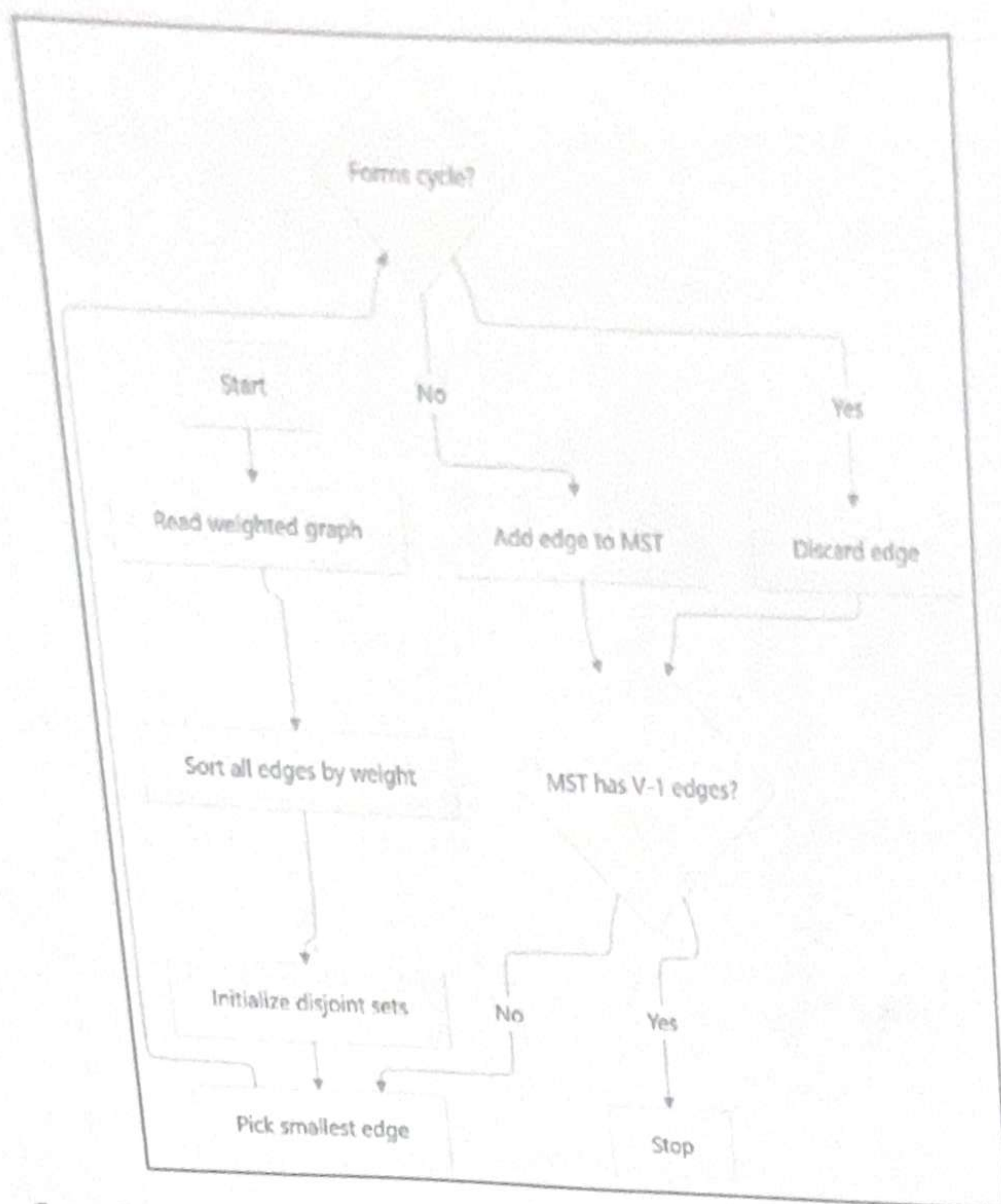
Flowchart Diagram:



2. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

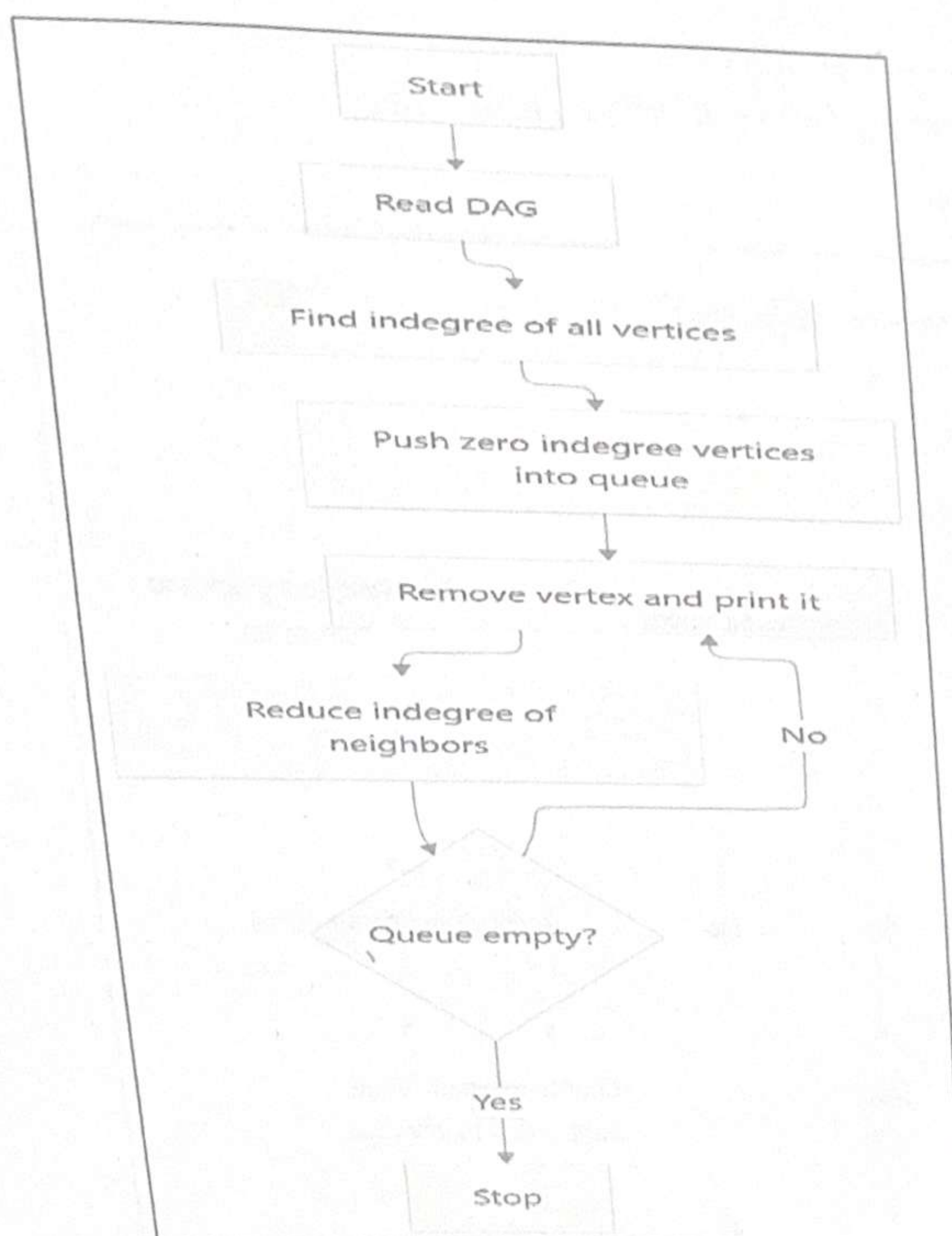
Flowchart Diagram:



Convert the flowchart for topological sort into code.

Flowchart Diagram:

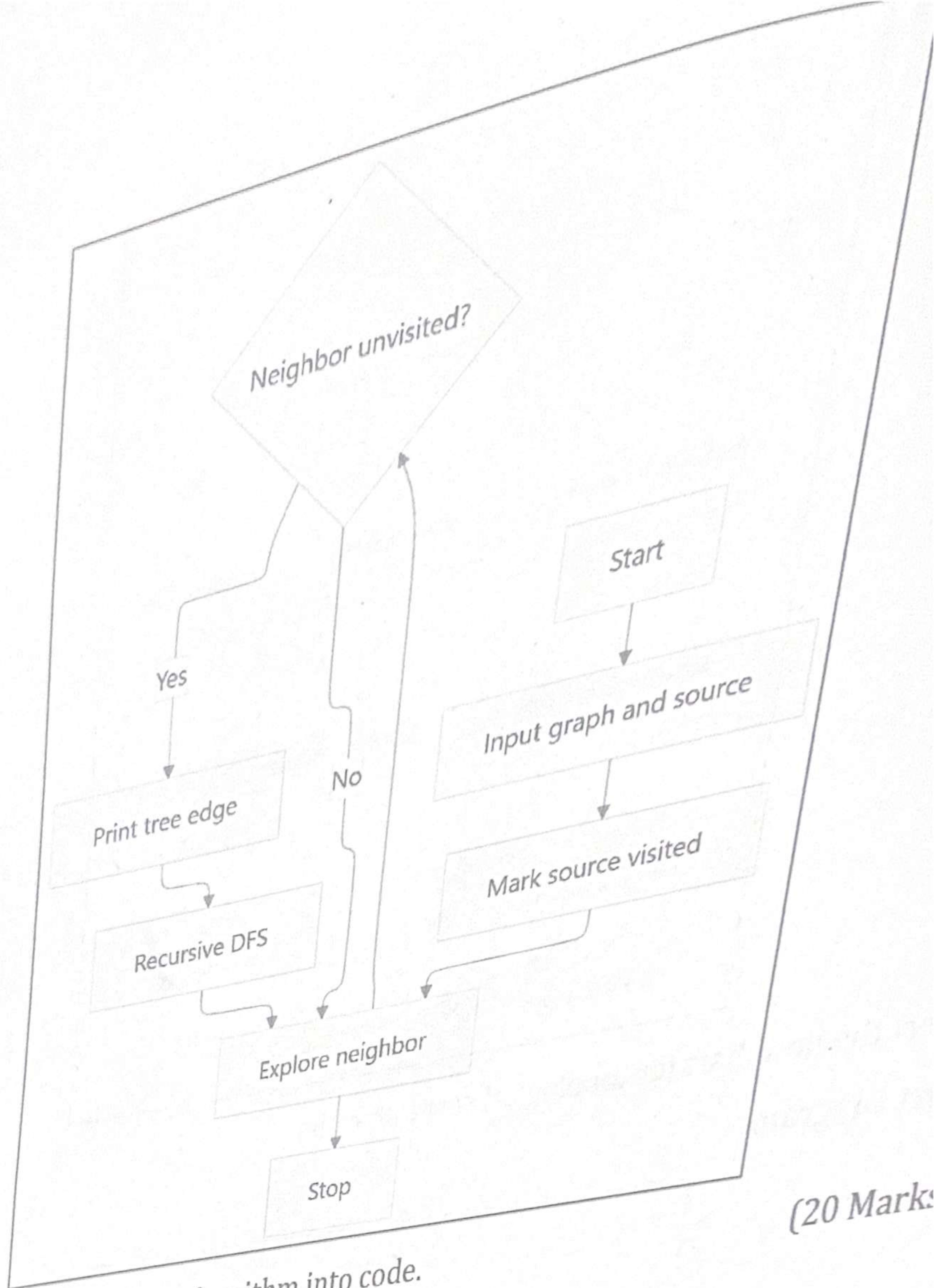
(20 Marks)



4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

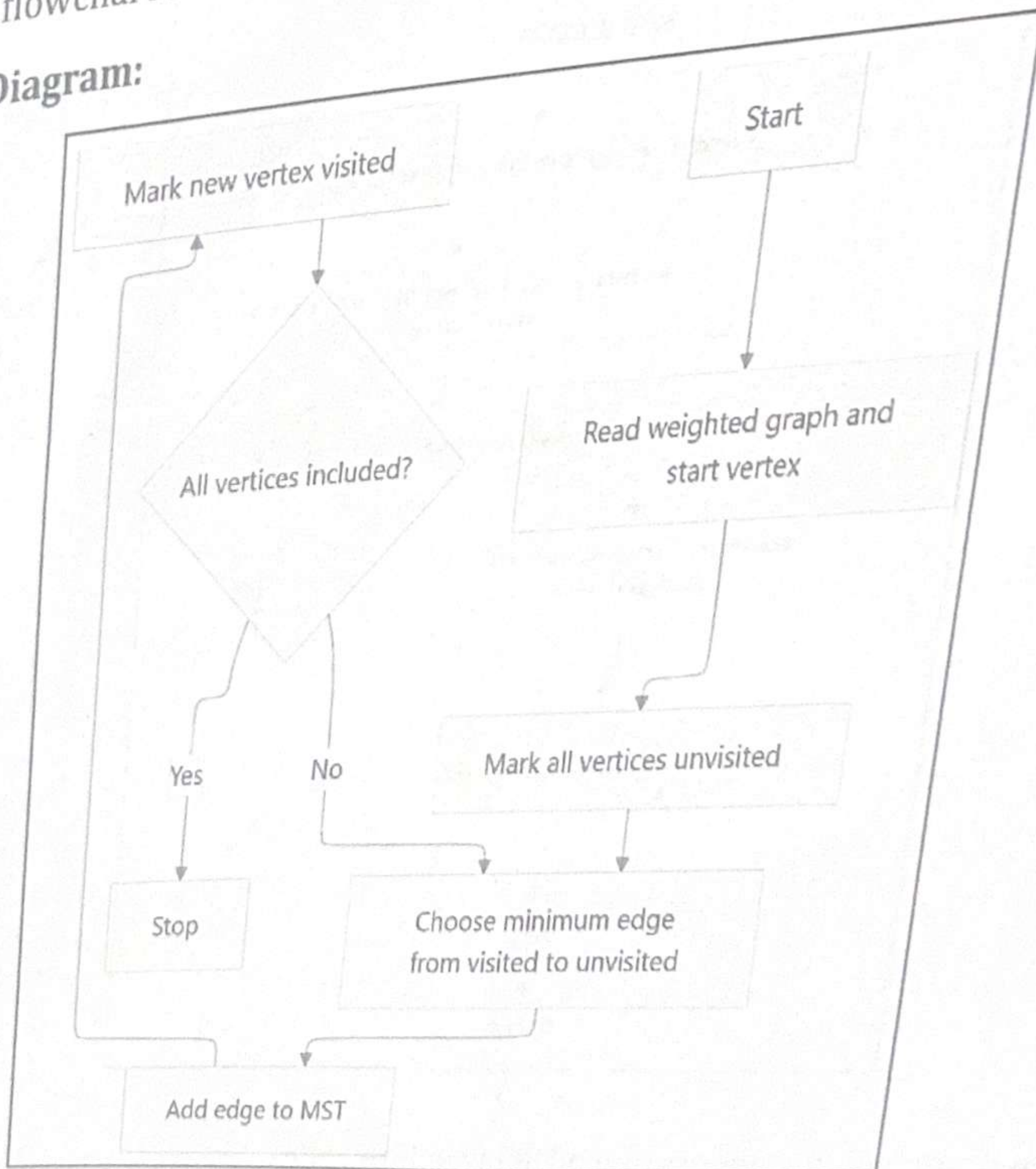
Flowchart Diagram:



(20 Marks)

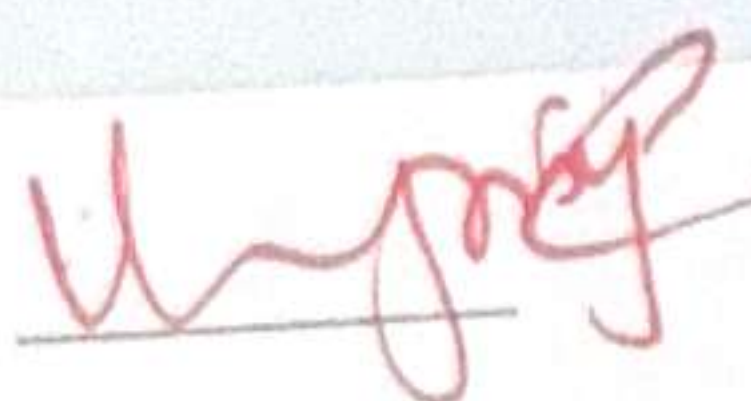
5. Convert the flowchart for Prim's algorithm into code.

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Breadth first Search (BFS)

```
#include <stdio.h>
```

```
int main () {
```

```
    int a[10][10], v[10] = {0}, q[10],
```

```
    int n, s, b = 0, r = 0, i, x;
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (int j = 0; j < n; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    scanf("%d", &s);
```

```
    v[s] = 1;
```

```
    q[r++] = s;
```

```
    printf("BFS");
```

```
    while (b < r) {
```

```
        n = q[b++];
```

```
        printf("%d ", n);
```

```
        for (i = 0; i < n; i++)
```

```
            if (a[n][i] && !v[i]) {
```

```
                v[i] = 1;
```

```
                q[r++] = i;
```

```
            }
```

```
        return 0;
```


2.) Centrals Algorithm:

#include <stdio.h>

struct edge {

int u, v, w;

} e[100] = {};

int p[100];

int find(int x) {

while (p[x] != x)

x = p[x];

return x;

} int main() {

int m, n, i; cost = 0, count = 0;

scanf("%d %d", &n, &m);

for (i = 0; i < m; i++)

scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].w);

for (i = 0; i < m; i++)

t = e[i].w; a[i] = e[i].u; b[i] = e[i].v;

}

if (a[i] == b[i])

printf("%d %d = %d\n", e[i].u, e[i].v, e[i].w);

cost += e[i].w;

p[a[i]] = b[i];

count++;

}

printf("cost = %d", cost);

return 0;

}

Topological Sort

```
#include <stdio.h>
```

```
int main () {
```

```
    int a[10][10]; int in[10] = {0}; int q[10], n;
```

```
    int b=0, x=0, i, j, x, count=0;
```

```
    scanf ("%d", &n);
```

```
    for (i=0; i<n; i++)
```

```
        for (j=0; j<n; j++) {
```

```
            scanf ("%d", &a[i][j]);
```

```
            if (a[i][j])
```

```
                in[j]++;
```

```
    printf ("Topological order: ");
```

```
    while (b < n) {
```

```
        x = q[b++];
```

```
        printf ("%d", x);
```

```
        count++;
```

```
    for (i=0; i<n; i++)
```

```
        if (a[x][i] && in[i] == 0)
```

```
            q[x++] = i;
```

```
    }
```

```
    if (count < n)
```

```
        printf ("In cycle graph");
```

```
    return 0;
```

```
}
```


4.) DFS Tree:

```
#include <stdio.h>
```

```
int a[100][100], v[100], n;
```

```
void DFS(int x){
```

```
    int i;
```

```
    v[x] = 1;
```

```
    printf("%d ", x);
```

```
    for(i=0; i<n; i++){
```

```
        printf("InTree Edge %d %d ", x, i);
```

```
        DFS(i);
```

```
    }
```

```
int main(){
```

```
    scanf("%d", &n);
```

```
    for(i=0; i<n; i++){
```

```
        for(j=0; j<n; j++){
```

```
            scanf("%d", &a[i][j]);
```

```
        }
```

```
    }
```

```
    printf("DFS:");
```

```
    return 0;
```

```
}
```


Prim's Algorithm :

```
#include <stdio.h>
```

```
#define INF 999
```

```
int main()
```

```
{  
    int a[10][10], v[10] = {0};
```

```
    int n, i, j, k, min_u, w, cost = 0;
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
            v[0] = 1;
```

```
    printf("MST: \n");
```

```
    for (k = 0; k < n - 1; k++)
```

```
        min = INF
```

```
        for (i = 0; i < n; i++)
```

```
            if (!v[i])
```

```
                for (j = 0; j < n; j++)
```

```
                    min = a[i][j];
```

```
                    u = i;
```

```
                    w = j;
```

```
            }
```


Printf ("1.d 1.d 2 1.n", u, w, min),

lost = min,

v(w) = 1,

{
Printf ("lost = 1.d", lost),
section 0,
}

Ugras